Chapter 1

IMPLEMENTATION

This section details the practical implementation of the closed-loop Security Orchestration, Automation, and Response (SOAR) architecture. It covers the deployment of the virtualized multi-node sandbox, the engineering of the evasive Python ransomware payload, the configuration of the real-time Snort network sensor, and the development of the automated cross-platform remediation pipeline.

1.1 Virtual Infrastructure Setup

The foundation of the simulation required deploying a strictly isolated, multi-node sandbox. Using Oracle VM VirtualBox, three distinct virtual machines were provisioned:

- Kali Linux (Attacker)
- Ubuntu Server (Watchtower/SOC)
- Windows 10 (Target)

A hypervisor-level "Host-Only" virtual network adapter was configured across all three nodes. This ensured that all subsequent malware execution, network beaconing, and SSH tunneling remained entirely contained within the virtual lab, completely neutralizing the risk of accidental host infection or external network exposure.

1.2 Target Environment Configuration

To accurately simulate an endpoint vulnerable to "Living off the Land" attacks, the Windows 10 target machine required specific administrative configurations.

- Windows Defender and active heuristic scanning were disabled via Group Policy.
- A designated dummy directory `C:\RansomwareLab\codes` was populated with sample files.
- The OpenSSH Server feature was installed and activated natively within Windows.

This configuration enabled the Watchtower node to remotely inject remediation commands later in the workflow.

1.3 Ransomware Payload Engineering (Offensive)

The offensive component of the lab involved engineering a custom Python ransomware script named `attack.py`. Utilizing the `cryptography.fernet` library, the payload was programmed to perform rapid directory traversal and apply AES 128-bit symmetric encryption to all files within the target directory while appending a `.locked` extension.
The script was designed to execute silently from a hidden directory

Upon successful encryption, the payload instantly broadcasted a plain-text Command and Control (C2) beacon over a TCP socket to signal a successful breach.

1.4 Network Sensor Configuration (Snort IDS)

With the payload engineered, the defensive detection layer was implemented on the Ubuntu Watchtower node.

Snort, an open-source Intrusion Detection System (IDS), was installed and configured to listen promiscuously on the Host-Only network interface. A custom rule was written within the `local.rules` file specifically designed to intercept the exact TCP plain-text signature broadcasted by the ransomware payload.

Snort was configured to immediately dump a high-priority alert into:

`/var/log/snort/alert_fast.txt`

the exact millisecond the C2 beacon crossed the network.

1.5 SOAR Daemon Development (Defensive)

The core of the automated defense lies in the Security Orchestration, Automation, and Response (SOAR) daemon.

A Python script was developed on the Ubuntu server to act as a continuous background watcher. Using file-tailing logic, this daemon monitored the Snort alert log in real time.

The script was programmed with conditional logic such that the moment a new ransomware alert was written to the log by Snort, the daemon instantly parsed the trigger and initiated the pre-programmed incident response playbook without requiring human intervention.

1.6 Automated Remediation Pipeline

Once triggered, the SOAR daemon entered the response phase.

The Python script dynamically generated a Windows PowerShell command specifically designed to locate the hidden payload and reverse the damage. Since the command orig-

inated from a Linux Bash environment but executed within Windows, the script handled necessary cross-platform translation and variable escaping to prevent syntax errors.

This generated command ultimately triggered a hidden `decrypt.py` script on the Windows machine, restoring the encrypted `.locked` files to their original state.

1.7 Cross-Platform Authentication (SSH)

For the pipeline to achieve a near-zero Mean Time to Respond (MTTR), the automated remediation commands had to be transmitted instantly to the target endpoint.

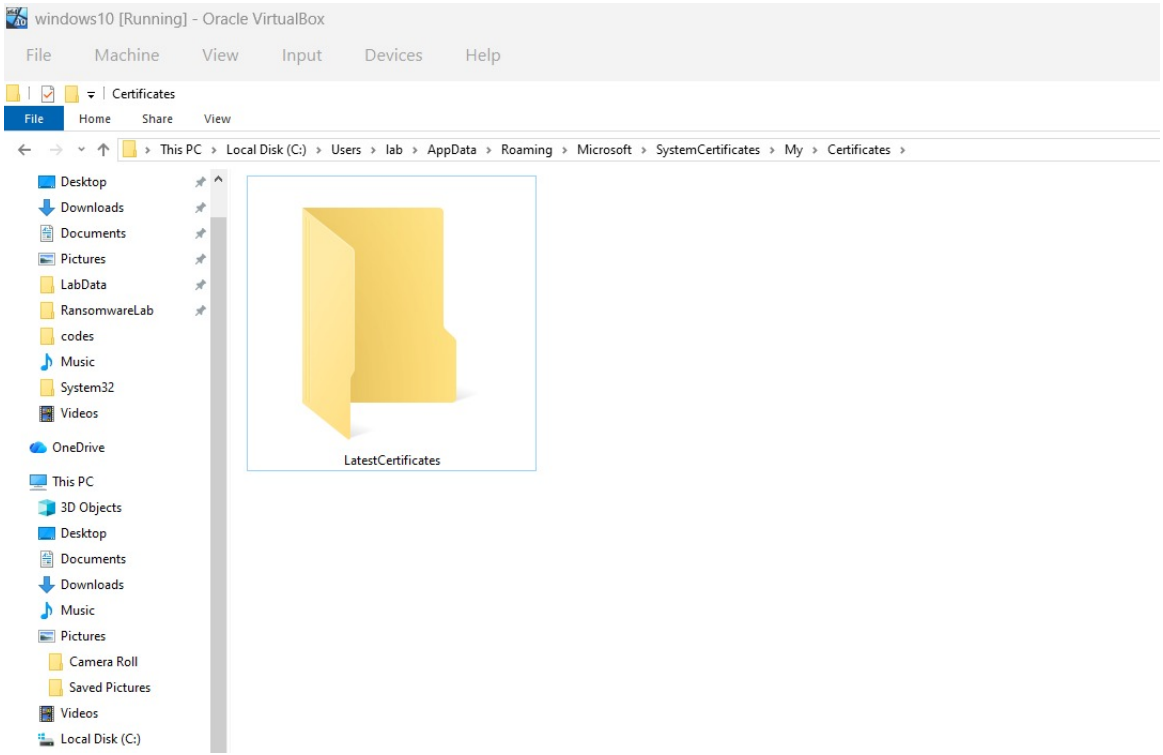
To accomplish this, the Linux utility `sshpass` was implemented within the SOAR daemon. This utility bypassed the standard interactive SSH password prompt, allowing the Ubuntu server to automatically authenticate into the Windows 10 machine.

By removing the need for manual credential entry, the system seamlessly tunneled PowerShell remediation commands at machine speed, significantly reducing incident response time.

1.8 Results

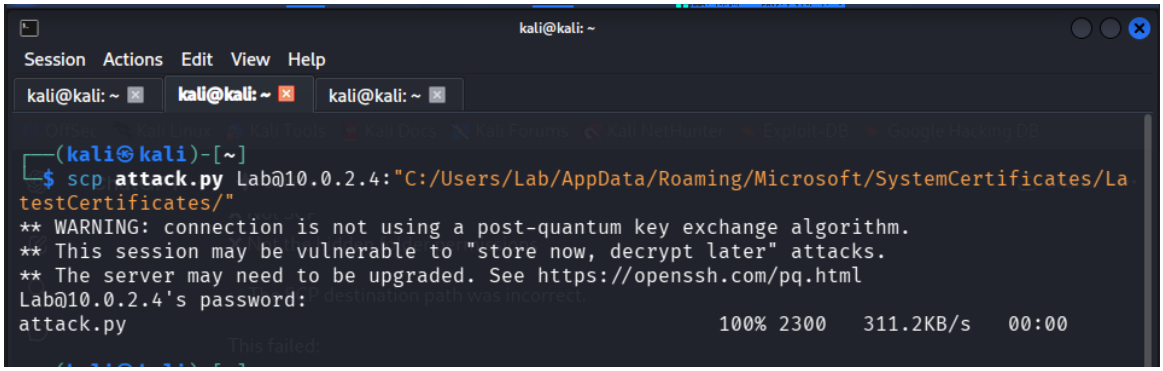
Phase 1: Payload Delivery Concealment

1. Target Directory Preparation Depicted is the Windows 10 File Explorer navigated deep into the `AppData\Roaming\Microsoft\SystemCertificates` path, where a new folder named LatestCertificates has been created. The working demonstrates the preparation of a stealthy drop zone. By nesting the payload inside a legitimate-looking system directory, the simulation mimics how Advanced Persistent Threats (APTs) hide malicious files from casual observation and routine antivirus scans.

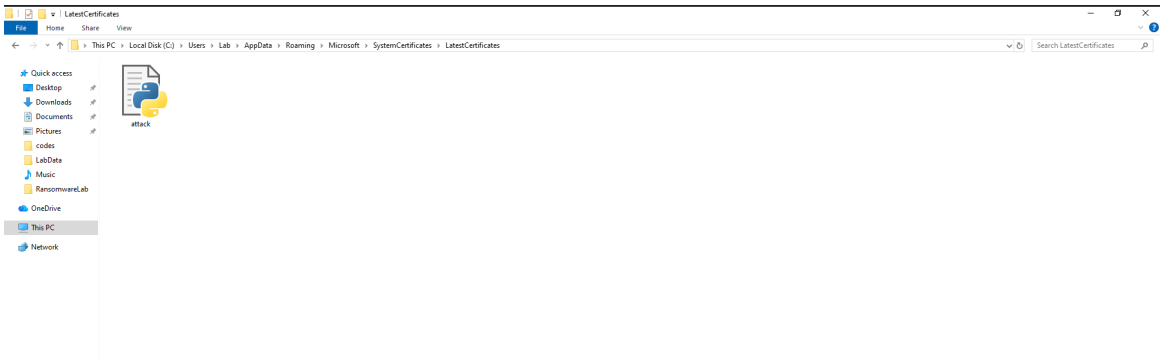


2. Payload Transmission This presents a Kali Linux terminal executing a Secure Copy Protocol (scp) command to transfer a file named attack.py. The working simulates an attacker smuggling a weaponized payload onto the victim machine. The scp command se-

curely sends the script over the network directly into the hidden LatestCertificates directory prepared earlier, bypassing basic network file transfer monitoring.

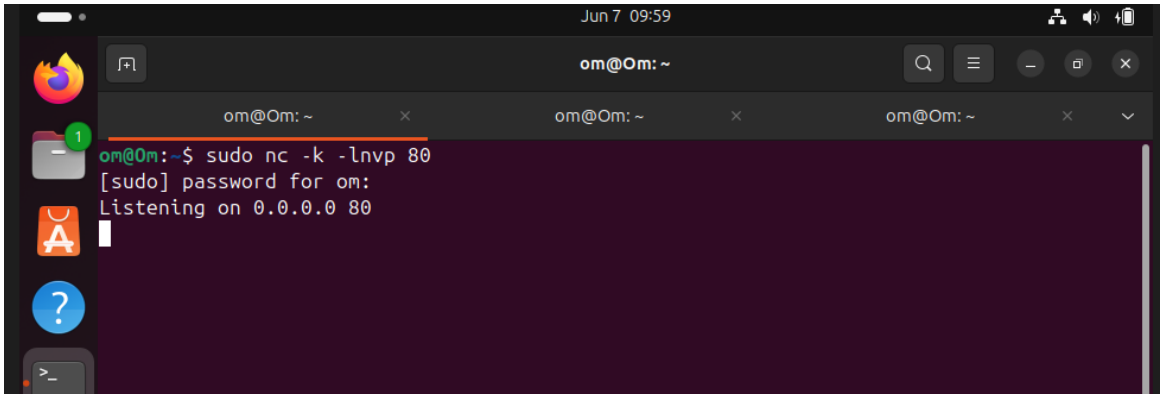


3. Delivery Confirmation Illustrated here is the Windows File Explorer opened inside the LatestCertificates folder, revealing the attack.py script now present on the victim machine. The working visually confirms that lateral movement or direct file transfer was successful. The payload is now resting passively in its concealed location, awaiting remote detonation without triggering immediate alarms.

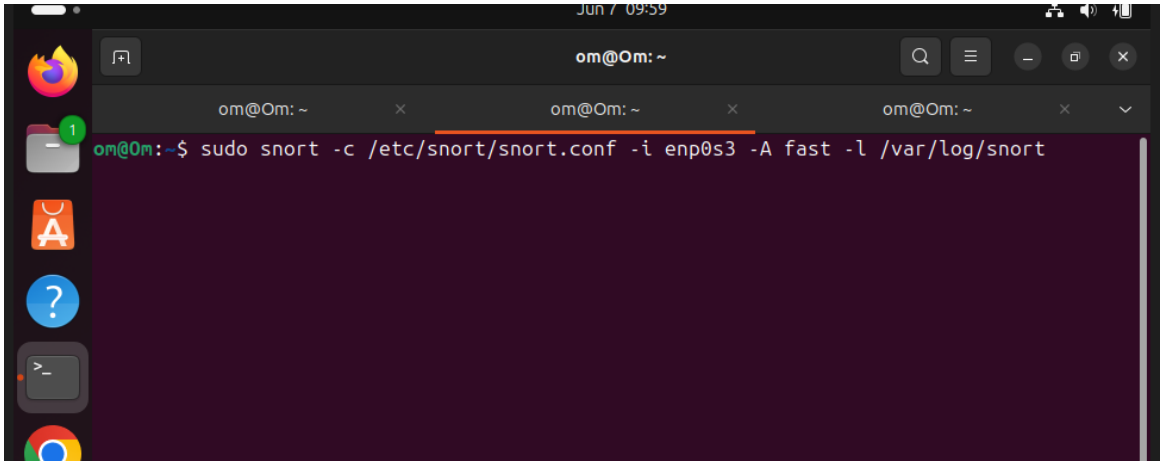


Phase 2: Defensive Infrastructure Setup

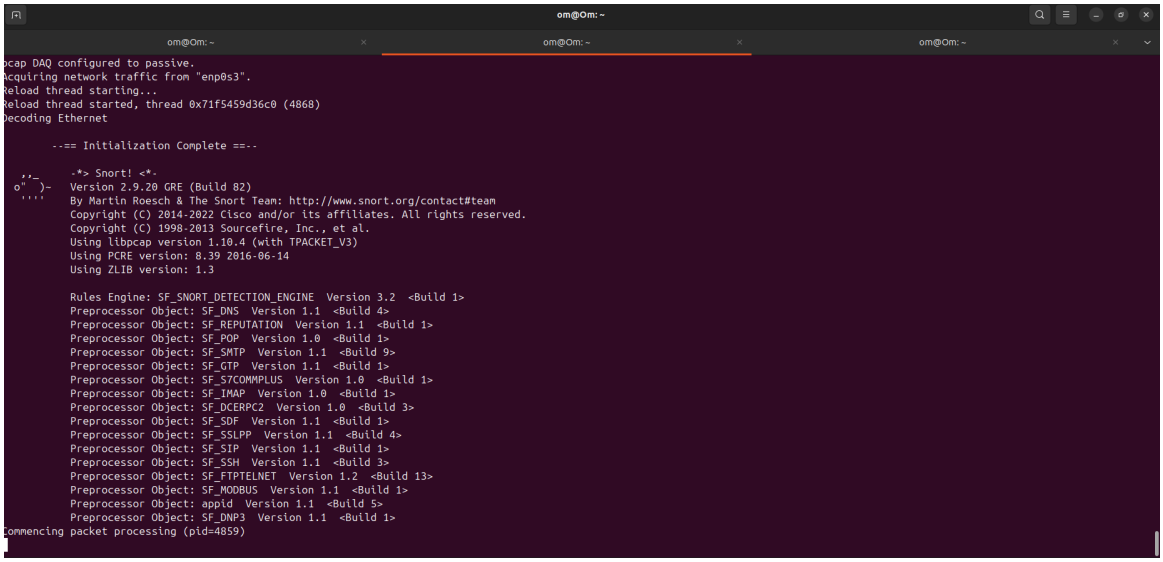
1. – Manual Listener Test The Ubuntu terminal executing a Netcat command (sudo nc -k -lnvp 80) is captured, which initializes a basic network listener on port 80. The working serves as a manual diagnostic tool before automated systems take over. By listening for incoming connections, the administrator verifies that the Ubuntu server can successfully receive network beacons from the Windows machine, ensuring the communication path is functional.



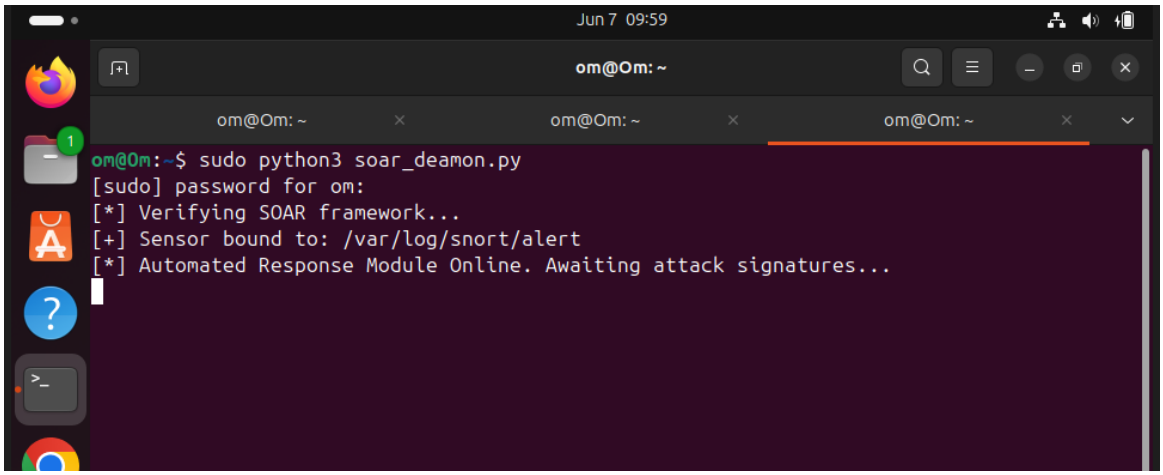
2. IDS Engine Execution The Ubuntu terminal executing the startup command for the Snort Intrusion Detection System is displayed. The command instructs Snort to boot up, read its configuration rules from snort.conf, actively sniff network traffic on the enp0s3 interface, and write any fast alerts to the /var/log/snort directory. This step activates the network monitoring layer of the defense.



3. IDS Initialization Verification The initialization log output of the Snort 2.9.20 engine as it loads is provided. The working confirms that the defensive perimeter is active and correctly configured. The log details Snort successfully loading its preprocessors and rules engines, meaning the system is now actively hunting for the ransomware's specific network signature.

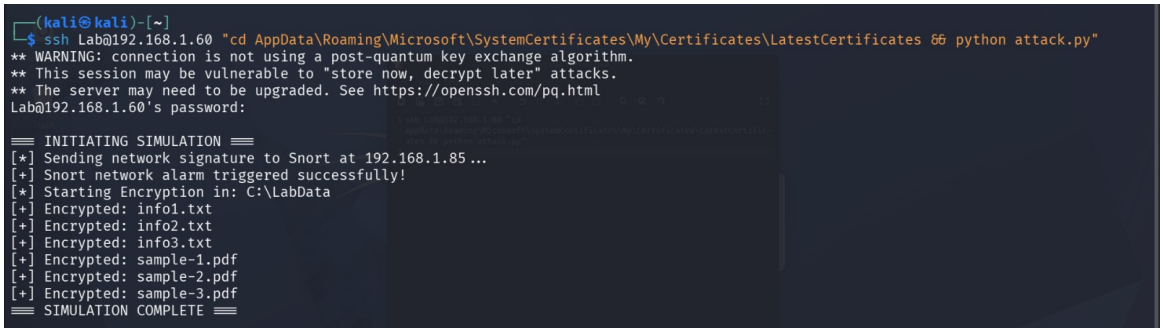


4. SOAR Daemon Activation The Ubuntu terminal running the soar_daemon.py Python script is presented, bringing the automated response module online. The working demonstrates the script successfully binding to the Snort alert log directory (/var/log/snort). The daemon then enters a waiting state, ready to trigger the remediation sequence the moment a threat signature is detected, closing the loop between detection and response.

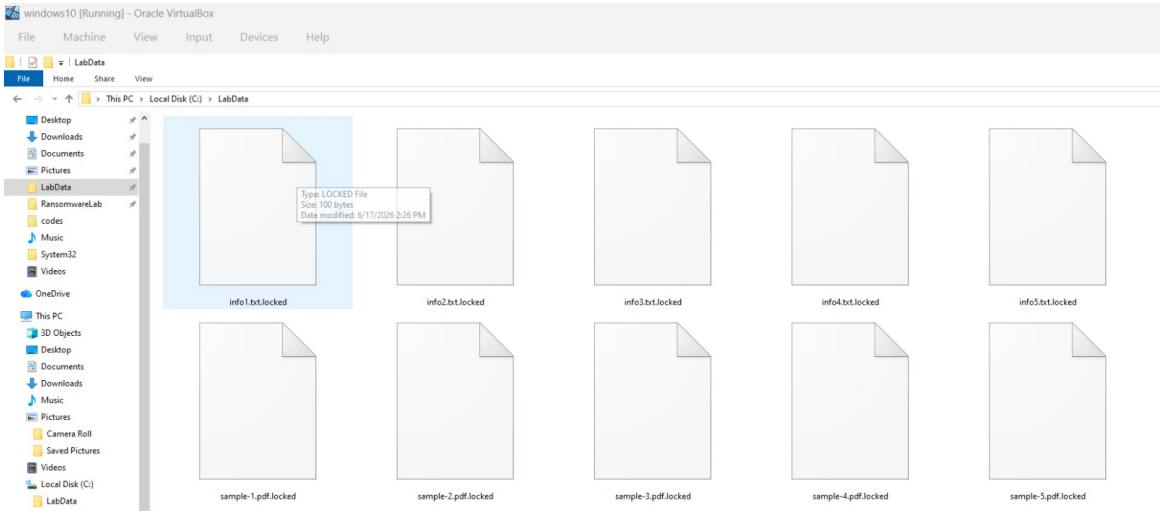


Phase 3: Detonation / Verification

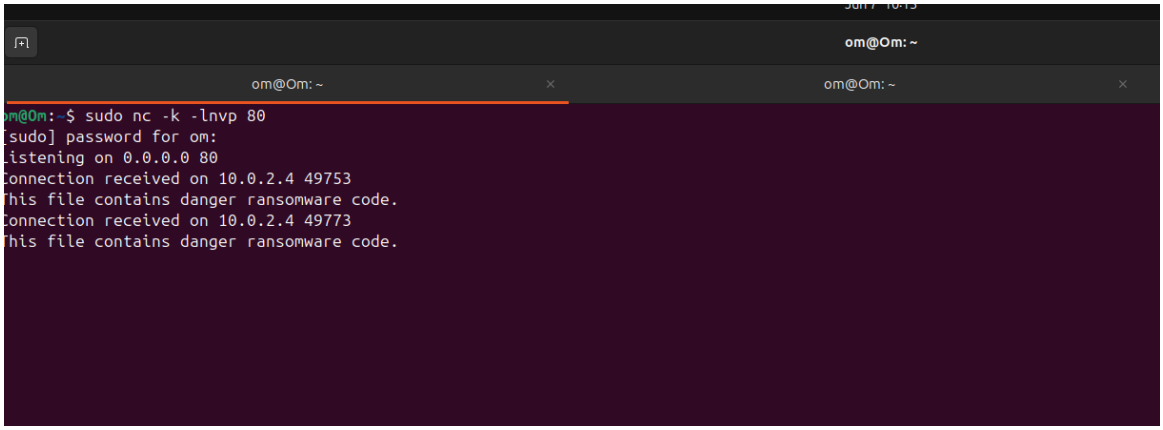
1. – Remote Payload Detonation A Kali terminal using SSH to remotely trigger the hidden attack.py script on the Windows machine is captured. The working represents the active attack phase: the script executes, immediately sends a network alarm beacon to the Ubuntu server, and then begins high-speed cryptographic locking of all .pdf and .jpg files inside the target `C:\LabData directory`.



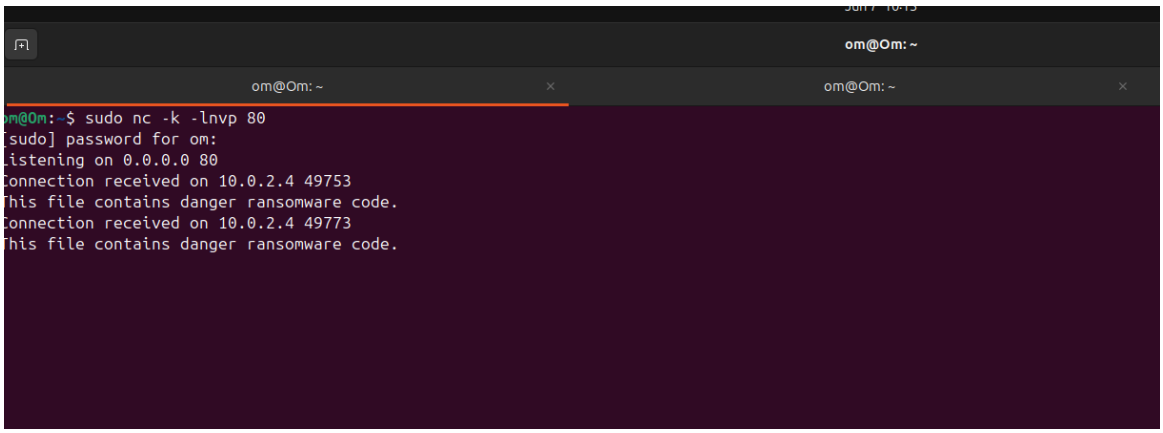
2. – Encryption Verification The Windows File Explorer displaying the C:\LabData directory immediately after the attack is shown. The working proves that local payload execution was successful. All previously normal files have been stripped of their original extensions and converted into files labeled as "LOCKED Files," visually simulating a total loss of data availability due to ransomware encryption.



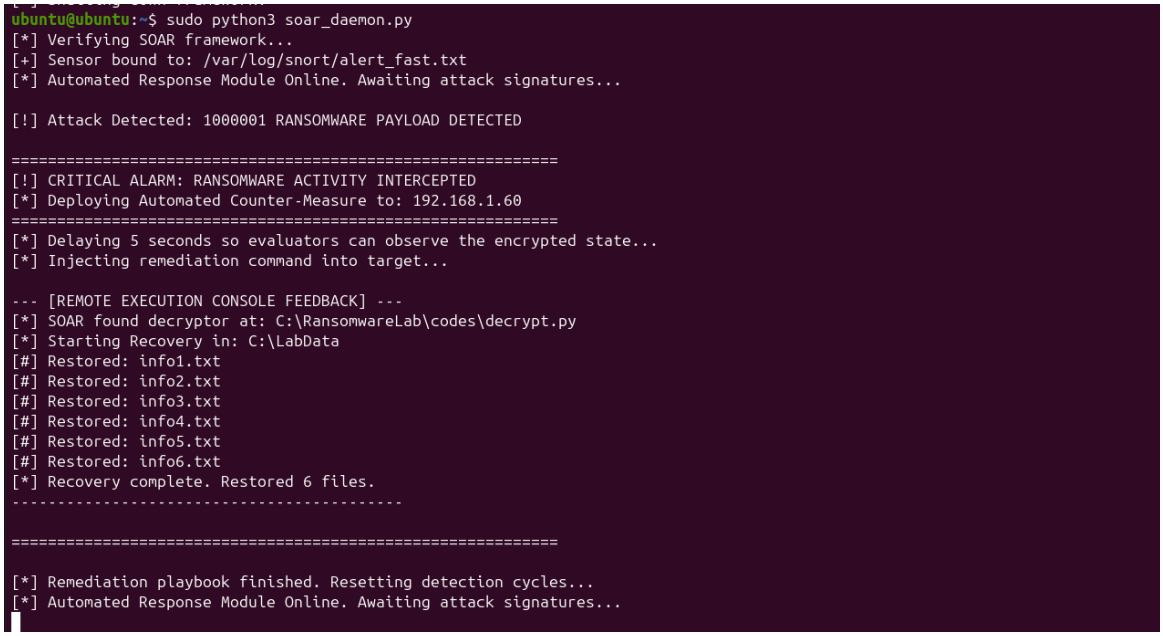
3. – Network Alarm Receipt The Ubuntu terminal running the Netcat listener (from step 4) displaying incoming text is illustrated. The working proves the attack script's secondary function succeeded: the Windows machine transmitted the hardcoded threat signature string ("This file contains danger ransomware code") across the network. This exact string is the trigger that the Snort and SOAR infrastructure relies on to initiate automated defense.



4. – SOAR Playbook Execution The Ubuntu terminal console output of the soar_daemon.py script as it reacts to the simulated attack is provided. The working demonstrates the core of automated defense: the daemon parses /var/log/snort/alert, detects the "Ransomware Signature Detected" event, and extracts the attacker's IP address (10.0.2.4). It then establishes an SSH connection back to the compromised Windows machine and triggers decrypt.py. The terminal confirms the cryptoviral effect is reversed, listing all 22 restored files.



5. – File Restoration Verification The Windows File Explorer displaying the C:\LabData directory immediately after the SOAR daemon's intervention is depicted. The working visually validates the ultimate success of the closed-loop architecture. All files have been stripped of their malicious "LOCKED" status and fully restored to their original .pdf and .jpg formats without any manual human intervention, proving the automated orchestration worked flawlessly.


ubuntu@ubuntu:~$ sudo python3 soar_daemon.py
[*] Verifying SOAR framework...
[+] Sensor bound to: /var/log/snort/alert_fast.txt
[*] Automated Response Module Online. Awaiting attack signatures...
[!] Attack Detected: 1000001 RANSOMWARE PAYLOAD DETECTED
[!] CRITICAL ALARM: RANSOMWARE ACTIVITY INTERCEPTED
[*] Deploying Automated Counter-Measure to: 192.168.1.60
[*] Delaying 5 seconds so evaluators can observe the encrypted state...
[*] Injecting remediation command into target...
--- [REMOTE EXECUTION CONSOLE FEEDBACK] ---
[*] SOAR found decryptor at: C:\RansomwareLab\codes\decrypt.py
[*] Starting Recovery in: C:\LabData
[#] Restored: info1.txt
[#] Restored: info2.txt
[#] Restored: info3.txt
[#] Restored: info4.txt
[#] Restored: info5.txt
[#] Restored: info6.txt
[*] Recovery complete. Restored 6 files.
[*] Remediation playbook finished. Resetting detection cycles...
[*] Automated Response Module Online. Awaiting attack signatures...